

# Personal Expense Tracker

A Python-Based Command-Line Application for Managing Personal Spending

Graduation Project Documentation

Prepared by: Senper Elias Tesfaye

Teferi Mekonnen Polytechnic College

Department: Web Development and Database Administration

Addis Ababa, Ethiopia

# 1. Project Title

Personal Expense Tracker: A Python-Based Command-Line Application for Managing Personal Spending

## 2. Introduction

This project presents a Python-based command-line application designed to help individuals track and manage their personal daily expenses. The system allows users to record expenses, view spending summaries, filter records by category, and permanently store their financial records using JSON-based file storage. It was built as a graduation project to demonstrate practical, beginner-to-intermediate level Python programming skills, including data structures, control flow, input validation, and file handling.

## 3. Background

Many individuals, particularly students, struggle to keep track of their day-to-day spending. Traditional methods such as writing expenses on paper or relying on memory are unreliable, difficult to search through, and prone to loss or damage. Existing solutions such as mobile banking apps or commercial budgeting software are often unavailable, too complex, or require an internet connection. This project was developed to address that gap using a simple, accessible, offline programming solution.

## 4. Problem Statement

Without a structured system, individuals often lose track of where their money goes, making it difficult to calculate totals, identify spending patterns by category, or review past financial decisions. There is a need for a lightweight, easy-to-use tool that allows users to record and review their expenses without requiring complex software, an internet connection, or specialized technical knowledge.

## 5. General Objective

To design and implement a Python-based Personal Expense Tracker that allows users to record, view, analyze, and manage their personal expenses efficiently.

## 6. Specific Objectives

- To allow users to add new expense records with amount, category, and notes
- To validate user input and prevent invalid or illogical data (e.g., non-numeric or negative amounts)
- To allow users to view all recorded expenses in a readable format
- To calculate and display total spending across all recorded expenses
- To allow users to filter and calculate total spending by a specific category
- To allow users to delete incorrect or unwanted expense records
- To ensure data persists between program sessions using JSON file storage

## 7. Scope

The system is limited to single-user, offline, command-line operation. It covers recording, viewing, totaling, filtering, and deleting personal expense records. It does not include multi-user accounts, budgeting/forecasting features, graphical charts, or a web/mobile interface in its current version, though these are identified as possible future improvements.

## 8. Target Users

The primary target users are students and individuals who want a simple, private way to track personal spending without needing an internet connection, a bank account link, or a paid subscription service.

## 9. System Features

- **Add Expense** — record an expense with amount, category, and an optional note
- **View All Expenses** — display every recorded expense in a readable list
- **View Total Spending** — calculate and display the sum of all recorded expenses
- **View Spending by Category** — filter and total expenses for a chosen category
- **Delete an Expense** — remove a specific expense record by its listed number
- **Persistent Storage** — automatically save and reload data using a JSON file, so records are not lost when the program closes

## 10. Technologies Used

- **Python 3** — core programming language for all program logic
- **Built-in json module** — used to save and load expense data in JSON format
- **Visual Studio Code** — code editor used for development
- **Command-line interface (terminal)** — used to run and interact with the program

## 11. System Design

The system follows a simple linear data flow: the user interacts with a text menu, the program processes the chosen action, and the result is either stored to or retrieved from a JSON file.

```
User --> Menu Choice --> Python Processing (validate / calculate / search)
      --> Data (expenses list <--> expenses.json file) --> Output to Screen
```

### 11.1 Data Structure

Each expense is represented as a Python dictionary with three keys: **amount** (a float), **category** (a string), and **note** (a string). All expenses are stored together in a single Python list, referred to as **expenses**. This list-of-dictionaries structure maps directly onto the JSON file format used for permanent storage.

```
expense = {"amount": 50.0, "category": "Food", "note": "Lunch"}
expenses = [expense, ...]
```

## 11.2 Program Flow

On startup, the program attempts to load any previously saved expenses from `expenses.json`. It then enters a continuous loop that displays a numbered menu and waits for the user's choice. Based on the choice, it calls the relevant logic (add, view, total, filter, delete) and returns to the menu, repeating until the user selects the Exit option.

## 12. Implementation

The application was implemented incrementally, one feature at a time, with each feature tested before moving to the next. Core implementation details include:

### 12.1 Input Validation

User-entered amounts are converted from text to a floating-point number using `float()`. This conversion is wrapped in a `try/except` block so that non-numeric input (such as letters or an empty entry) does not crash the program. A second validation layer checks that the amount is strictly greater than zero, since a zero or negative value does not represent a logical expense.

```
try:
    amount = float(input("Enter amount: "))
    if amount <= 0:
        print("Amount must be greater than zero")
    else:
        # proceed to record the expense
        ...
except:
    print("Invalid amount. Please enter a number.")
```

### 12.2 Data Persistence

The program uses Python's built-in `json` module to save the expenses list to a file (`expenses.json`) every time an expense is added or deleted, and to load that file back into memory when the program starts. If no file exists yet (first run), the load function safely returns an empty list instead of causing an error.

```
def load_expenses():
    try:
        with open("expenses.json", "r") as file:
            return json.load(file)
    except:
        return []

def save_expenses():
    with open("expenses.json", "w") as file:
        json.dump(expenses, file)
```

### 12.3 Category Filtering

The 'View Spending by Category' feature loops through all stored expenses, comparing each expense's category to the user's search term, and accumulates a running total for matches. A boolean flag variable (`found`) tracks whether any matching expenses were located, so the system can give clear feedback if the category does not exist in the records.

## 12.4 Safe Deletion

The delete feature displays every expense alongside a visible index number, then asks the user which number to delete. Before removing anything, the program checks that the entered number is within the valid range of existing expenses, preventing the program from crashing on an out-of-range or invalid value.

## 13. Testing

Each feature was tested systematically against normal, invalid, empty, and boundary input cases. The table below summarizes the test cases performed and their outcomes.

| Feature          | Test Case             | Input Example                 | Result                                        |
|------------------|-----------------------|-------------------------------|-----------------------------------------------|
| Add Expense      | Normal input          | amount=30, category=Transport | Added successfully                            |
| Add Expense      | Invalid (non-numeric) | amount="abc"                  | Rejected with error message                   |
| Add Expense      | Empty input           | amount=(blank)                | Rejected with error message                   |
| Add Expense      | Boundary (zero)       | amount=0                      | Rejected — must be greater than zero          |
| Add Expense      | Negative value        | amount=-20                    | Rejected — must be greater than zero          |
| View All         | Empty list            | no expenses yet               | "No expenses recorded yet."                   |
| View All         | Populated list        | 1+ expenses stored            | All expenses displayed correctly              |
| View Total       | Empty list            | no expenses yet               | "No expenses recorded yet."                   |
| View Total       | Populated list        | multiple expenses             | Correct sum displayed                         |
| View by Category | Match found           | search="Food"                 | Correct category total shown                  |
| View by Category | No match              | search=nonexistent category   | "No expenses found in that category."         |
| Delete           | Empty list            | no expenses yet               | "No expenses recorded yet."                   |
| Delete           | Valid index           | existing index number         | Expense removed and saved                     |
| Delete           | Out-of-range index    | index=99                      | "Invalid number. No expense deleted."         |
| Delete           | Non-numeric input     | index="abc"                   | "Invalid input. Please enter a valid number." |
| Persistence      | Program restart       | close and reopen program      | Previously saved data reloaded correctly      |

## 14. Results

The final program successfully meets all specific objectives defined for this project. It allows a user to add, view, total, filter, and delete personal expense records through a simple menu-driven interface, with all data automatically saved and reloaded between sessions. All test cases in Section 13 produced the expected results, including correct handling of invalid, empty, and boundary input, demonstrating that the system behaves reliably under both normal and unexpected usage conditions.

## 15. Limitations

- The system runs only in a command-line interface; it does not currently have a graphical or web-based interface
- Category search is case-sensitive (e.g., "Food" and "food" are treated as different categories)
- The system supports a single user with no login or account separation
- Expenses do not currently include an automatic date/timestamp field
- Data is stored in a single local JSON file rather than a full database system

## 16. Future Improvements

- Make category search case-insensitive for a more forgiving user experience
- Automatically record the date of each expense using Python's datetime module
- Add sorting options (e.g., by amount or date)
- Migrate storage from a JSON file to a relational database such as SQLite or MySQL for larger datasets
- Develop a web-based interface (e.g., using Flask) so the tracker can be used through a browser
- Add data visualization (charts/graphs) to show spending trends over time

## 17. Conclusion

This project demonstrates the design and implementation of a functional, well-tested Python command-line application that solves a genuine, everyday problem: keeping track of personal expenses. Through incremental development, careful input validation, and systematic testing, the system was built to be both reliable and understandable. Beyond the working software itself, the project reflects a solid, hands-on understanding of core Python concepts including variables, conditionals, loops, functions, lists, dictionaries, exception handling, and file-based data persistence — the foundation for further growth as a developer.